

HACKATHON PROJECT REPORT

DeviceGuardian AI

"Predict device failures before they happen."

Amudieshwar A G

Project vision, system architecture & complete workflow

AI-powered predictive maintenance platform

01 Problem statement

Laptops and smartphones have become essential to daily work, education, and communication — yet they still fail unpredictably, through battery degradation, overheating, storage failure, or sustained heavy use. Existing tools such as Lenovo Vantage, Samsung Device Care, HWMonitor, and Windows Task Manager only report the device's *current* status: CPU usage, battery percentage, RAM usage, temperature, storage usage.

None of them can answer the questions that actually matter to a user or an IT department:

- Will my SSD fail in the next few months?
- Why is my battery degrading faster than normal?
- Is my laptop continuously overheating?
- Which component should be serviced first?
- What preventive maintenance would avoid a costly repair?

Left unanswered, this leads to unexpected failures, higher repair costs, warranty claims, data loss, device downtime, and dissatisfied customers. The gap isn't monitoring — it's **foresight**.

02 Proposed solution

DeviceGuardian AI is an AI-powered Device Health Intelligence platform. Rather than simply displaying hardware metrics, it continuously analyzes device telemetry, predicts failures before they occur, explains *why* a prediction was made, and recommends concrete preventive actions.

Positioning matters here: Instead of claiming overall from multiple specialized AI models — one for battery, one for storage, one for thermal behaviour. This is both more technically defensible and easier to demonstrate convincingly within a hackathon timeframe. the platform is framed as calculating an

Target devices — hackathon MVP

Windows laptop

A lightweight monitoring agent installed on the machine collects detailed hardware telemetry every 30 seconds.

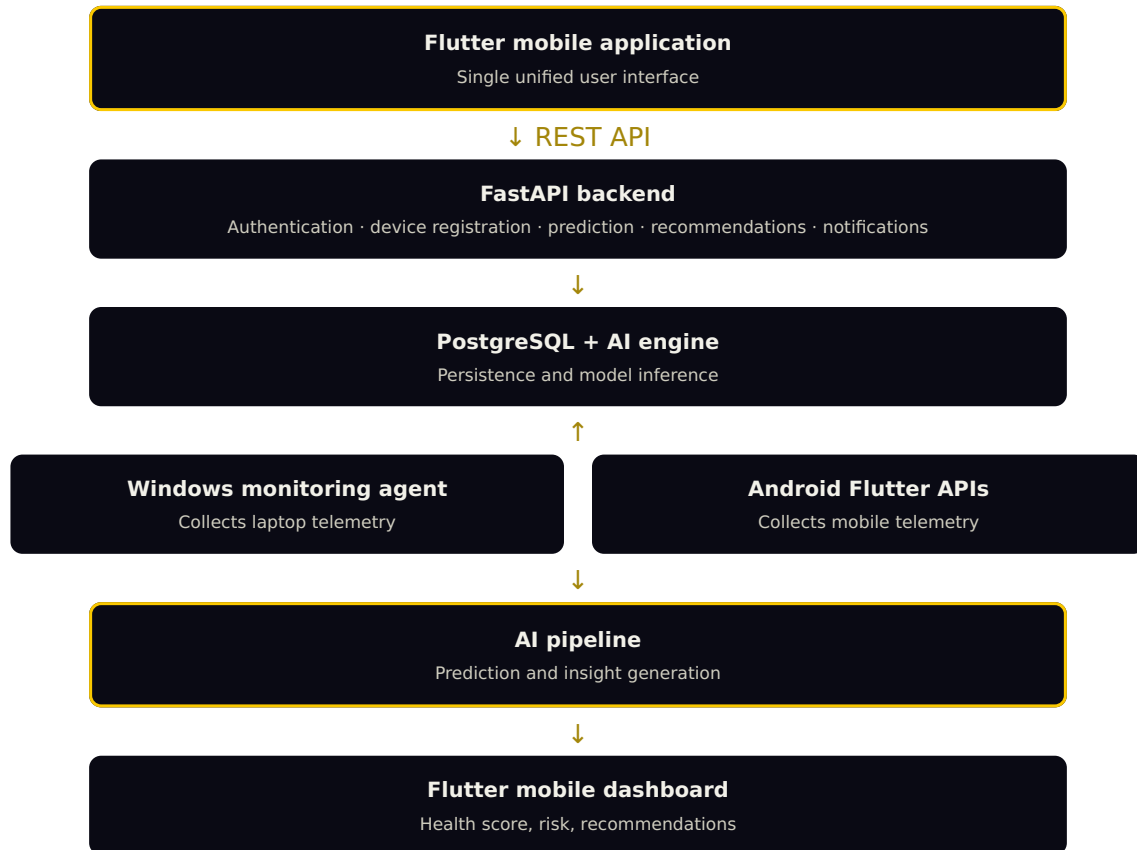
Android smartphone

Flutter uses native Android APIs to collect supported device data directly — no separate agent required.

Architecture is modular by design, leaving room to extend to macOS, Linux, smart TVs, printers, routers, and other IoT devices in future iterations.

03 System architecture

A single Flutter application is the unified interface for both device types. It talks to a FastAPI backend over REST, which in turn is backed by PostgreSQL and the AI engine. Telemetry reaches the backend through two different front doors — a Windows monitoring agent, or Flutter's own Android APIs — and converges into one AI pipeline.



04 Complete user workflow

The system behaves differently depending on whether the user connects a Windows laptop or an Android phone. Both converge on the same backend and dashboard.

Windows laptop workflow

- 1 Install the Flutter app**
User installs DeviceGuardian AI from the Play Store. With no devices connected yet, the home screen shows "No Devices Connected" and a "+ Add Device" action.
- 2 Choose device type**
Tapping "+ Add Device" prompts a choice: Windows laptop or this Android phone.
- 3 Guided laptop connection**
Flutter displays setup instructions — download the agent, install it, log in with the same account, and the laptop appears automatically. Flutter itself never installs the agent; it only guides the user.
- 4 Install the monitoring agent**
The user installs `DeviceGuardianAgent.exe`, which is configured to auto-start with Windows.
- 5 Log in to the agent**
The same DeviceGuardian account is used to log in from the agent, letting the backend associate this specific laptop with the user.
- 6 Collect live telemetry**
Every 30 seconds, the agent reads CPU usage, RAM usage, CPU temperature, battery health and capacity, SSD SMART health, disk errors, storage usage, running processes, and fan speed (where supported) via `psutil`, `WMI`, `pywin32`, and `LibreHardwareMonitor`.
- 7 Send telemetry to the backend**
The agent serializes the reading as JSON and posts it to FastAPI.

```
{
  "deviceId": "Laptop001",
  "cpu": 42,
  "ram": 63,
  "battery": 91,
  "temperature": 44,
  "ssd": 96
}
```

- 8 AI pipeline runs**
Telemetry is cleaned, engineered into features, scored by XGBoost, checked for anomalies by Isolation Forest, explained with SHAP, and converted into recommendations — producing overall health, failure risk, and battery / SSD / thermal risk sub-scores.
- 9 Dashboard displays the result**
Flutter calls `GET /deviceHealth` and renders overall health (e.g. 94%), battery (91%), SSD status (healthy), and an overall risk label (e.g. low risk).

Android phone workflow

Android does not require a monitoring agent — Flutter collects telemetry directly through native APIs after the user grants battery, storage, usage-statistics, and notification permissions.

Step	What happens
1 – 2	Install the app and log in.
3	Tap "+ Add Device" → "This Android Phone".
4	Flutter requests battery, storage, usage-statistics, and notification permissions.

5	Flutter collects battery percentage, battery temperature, charging status, storage usage, and RAM usage directly.
6	Telemetry is sent to FastAPI.
7	Backend runs prediction → SHAP → recommendations, the same pipeline used for laptops.
8	Flutter displays overall health (e.g. 96%), battery status, and risk level.

05 AI pipeline in detail



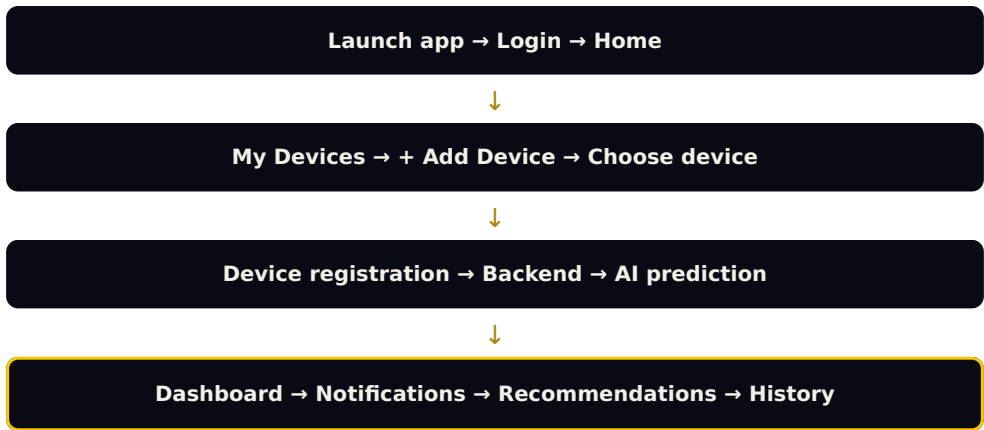
WORKED EXAMPLE

Failure risk: **81%** — battery wear 32%, CPU temperature 25%, SSD errors 18% (SHAP contribution breakdown).
Recommendations: back up the SSD, reduce heavy gaming sessions, clean the cooling fan, avoid overnight charging.

Why feature engineering matters

Feeding the model a single reading like "CPU temperature = 45°C" gives it almost nothing to work with. The backend instead computes average CPU temperature, temperature trend over time, battery drain rate, SSD error growth, and RAM utilization trend — these derived, time-aware features are what actually make the prediction meaningful.

06 Flutter application flow



Screen	Contents
My Devices	List of connected devices with a live health percentage each, e.g. Lenovo IdeaPad Slim 5 — 94%, Samsung Galaxy S24 — 96%.
Laptop dashboard	Overall health, CPU, RAM, temperature, battery, SSD status, and a prediction label (e.g. low risk).
Android dashboard	Overall health, battery, storage, temperature, and prediction label.
AI explanation ("Why?")	SHAP contribution breakdown — battery wear, CPU temperature, SSD errors, each as a percentage of the overall risk.
Recommendations	Actionable checklist: back up SSD, clean cooling fan, reduce overnight charging, etc.

07 Training data & datasets

The AI is **not** trained on the user's own device. Initial models are trained entirely on public, real-world research datasets:

Dataset	Used for
NASA Battery Dataset	Battery aging and remaining-useful-life cycles (voltage, current, temperature, capacity fade).
CALCE Battery Dataset (Univ. of Maryland)	Cross-validation for battery degradation patterns.
Backblaze SMART Dataset	Millions of real hard-drive/SSD SMART records with failure labels — reallocated sectors, read/write errors, power-on hours, temperature.
AI4I 2020 Predictive Maintenance Dataset	General predictive-maintenance concepts; industrial rather than laptop-specific, but useful for demonstrating the pipeline.

Once deployed, the monitoring agent continuously collects telemetry from real devices, letting an organization build its own device-specific historical dataset and periodically retrain the models — the same lifecycle used by production predictive-maintenance systems.

08 Technology stack

Component	Technology
Mobile application	Flutter
Backend	FastAPI
Database	PostgreSQL
Laptop monitoring agent	Python
System libraries	psutil, WMI, pywin32, LibreHardwareMonitor
AI model	XGBoost
Anomaly detection	Isolation Forest
Explainable AI	SHAP
Notifications	Firebase Cloud Messaging

09 Why this is different

Existing applications	DeviceGuardian AI
Show current device status	Predict future device failures
Reactive maintenance	Preventive maintenance
Basic diagnostics	AI-powered risk prediction
No explanation	SHAP-based explainability
No recommendations	Personalized maintenance suggestions
No learning from telemetry	Builds a historical telemetry dataset for continuous improvement

10 Why this works as a 24-hour hackathon project

- Technically feasible within 24 hours — no component requires a novel research contribution.
- Uses real telemetry rather than only simulated data, which is visibly more credible in a live demo.
- Cleanly separated responsibilities: the Windows agent collects laptop telemetry, Flutter is the unified interface and Android telemetry collector, FastAPI is the backend, and the AI pipeline handles prediction, anomaly detection, and explainability.
- Modular architecture that can scale to more device types later without a redesign.

11 Demo script — key talking points

Ready-made lines for walking judges through the product and pre-empting the questions they're most likely to ask.

"Existing tools like Task Manager or Device Care only tell you what's happening right now. DeviceGuardian AI tells you what's going to happen — before it costs you a repair, a warranty claim, or your data."

"We trained our initial models using public research datasets — NASA and CALCE for battery degradation, Backblaze for SSD failure prediction. Once deployed, DeviceGuardian's monitoring agent continuously collects live telemetry from real devices. This lets any organization build its own device-specific historical dataset over time and periodically retrain the models — exactly how production predictive-maintenance systems evolve."

"Instead of one model trying to predict every possible failure, we calculate a Device Health Score from multiple specialized models — one for battery, one for storage, one for thermal behaviour — each explained with SHAP so the recommendation is never a black box."

GUARDRAILS FOR THE LIVE DEMO

Avoid claiming prediction of motherboard or CPU failure — no usable public dataset backs that claim; stay within battery, SSD, and thermal, which are genuinely data-backed. **important**

Don't attempt to train on your own 20-minute telemetry sample live — train offline beforehand on the public datasets, and frame live telemetry purely as inference input and future retraining fuel.